Knowledge-Based Systems 309 (2025) 112896

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

RVFL-LSTM: A lightweight model with long-short term memory for time series

Qiang Liu [a], Qin Wang [a], Xizhao Wang [a,b,*]

[a] Big Data Institute, College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, 518060, China
[b] The Guangdong Key Laboratory of Intelligent Information Processing, Shenzhen University, Shenzhen, 518060, China

ARTICLE INFO

Keywords:
Time series prediction
Auto-regression
Long short-term memory (LSTM)
Random Vector Functional Link (RVFL)
Online sequential extreme learning machine (OS-ELM)

ABSTRACT

Neural networks have been widely used for time series prediction due to their excellent ability to capture the sequential relationship between the past and the future in time series data. However, the existing neural networks, such as long-short term memory(LSTM) and recurrent ones, are often criticized for their complex structure and strict training mode, but some lightweight network models often cannot achieve satisfying prediction performance. In order to tackle this challenge, this paper proposes a lightweight model with long-short term memory for time series, named RVFL-LSTM, which is trained for moving auto-regression task. To highlight the long and short-term patterns in time series data, RVFL-LSTM adjusts the input weights to learn the short-term patterns and updates the output weights gradually to capture the long-term patterns. Experimental results show that the proposed method can capture the long-term and short-term patterns efficiently, and it is very competitive in time series prediction with respect to prediction accuracy and computational complexity.

1. Introduction

A time series is formally defined as a sequence of random variables arranged in time order. Time series analysis is devoted to extracting meaningful summary and statistical information from the series. The research on time series analysis involves a lot of areas and topics, such as temperature prediction [1], air quality assessment [2], clinical medicine dispatching [3], electroencephalogram evaluation [4], stock market prices [5], security defense [6] and so on.

Time series prediction often comes down to a problem of causality: How did the past influence the future? Various methods have been developed to address that problem. Traditional techniques, such as the Auto-regressive Moving Average (ARMA) [7] and Auto-regressive Conditional Heteroskedasticity (ARCH) [8], are commonly employed to model time series data. These methods are effective, but they are often time-consuming while determining the model order, performing model diagnostics, and optimizing model parameters. Furthermore, since real-time series data originate from diverse sources and exhibit various characteristics, there is no single best approach for general time series modeling.

As continuous monitoring and data collection become more commonplace, the demand for effective analysis on time series using machine learning techniques is steadily rising [9]. In contrast to the classical methods that require the time series to be a stationary process, neural networks as time series analysis tools relax those assumptions. The recurrent neural network (RNN) [10] adds connections that feed information from hidden layers back into themselves and has been widely used for time series prediction. However, standard RNN architectures are difficult to train due to the vanishing gradient problems [11]. Long short-term memory (LSTM) [12,13] architecture was developed to address these problems. LSTM employs gated mechanisms to train the parameters within these memory components using the gradient descent algorithm, and the inputting data order during the forward and backward passes of LSTM is very strict.

Although some reported results on training and testing are promising, challenges persist in the design and implementation of LSTM. The complicated structure and the necessity of strict sequential computation during backpropagation (BP) introduce computational complexity issues. Additionally, the opaqueness of the training hinders the understanding of model [14,15].

Consequently, building an efficient model like LSTM for time series prediction is rather challenging. One potential approach to overcoming the challeng is to consider adopting the Random Vector Functional Link (RVFL) neural network [16]. RVFL [17] is a Single-hidden Layer Feed-forward Neural network (SLFN) and the input layer in RVFL connects directly to both the hidden and the output layer.

RVFL offers several advantages, including straightforward hardware implementation, rapid convergence, and compliance with the demands

* Corresponding author at: Big Data Institute, College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, 518060, China.
E-mail address: xizhaowang@ieee.org (X. Wang).

https://doi.org/10.1016/j.knosys.2024.112896
Received 29 December 2023; Received in revised form 18 November 2024; Accepted 15 December 2024
Available online 24 December 2024

of real-time applications [18]. The universal approximation capability of RVFL has been proven in theory [19,20]. Based on RVFL, Huang et al. [21] conducted further research and introduced a new neural network called Extreme Learning Machine (ELM). As a variant of RVFL, ELM eliminates connections from the input to the output layers. Building on this work, a great number of related theories and applications have been developed.

A number of ELM references for time series prediction are available currently, and we list some of the typical ones. Grigorievskiy et al. [22] applied the Optimally Pruned Extreme Learning Machine (OP-ELM) to the challenge of time series prediction and made a comparison among three known strategies: Recursive, Direct, and DirRec. Their experimental results revealed no superior strategy and each had its own merits. Ma et al. [23] proposed the Pruned Stacking Extreme Learning Machine (PS-ELM) algorithm for time series prediction, which introduced the ensemble pruning paradigm to improve the prediction performance. Xue et al. [24] employed the $\ell_{2,1}$-norm and Random Fourier Mapping-based Extreme Learning Machine ($\ell_{2,1}$ RF-ELM) for financial time series prediction. They compared $\ell_{2,1}$ RF-ELM with several other ELM variants and found that ELM efficiently and swiftly accomplished the tasks.

In the above-mentioned studies, ELM and its variations served as standard regressors and yielded favorable results. However, they are not primarily designed to capture the essential causality between past and future in time series prediction. These studies, in fact, overlook the long-short term patterns in time series. Online Sequential Extreme Learning Machine (OS-ELM) [25,26] addresses this issue through designing an online learning algorithm that inherits most advantages of ELM including robust generalization ability and rapid training speed. In OS-ELM, the network are initialized computed based on the training samples. Along with new training samples are added to the dataset, the corresponding output weights will be updated based on the initial network weights. It is worth noting that OS-ELM strictly requires that the number of training samples must exceed the number of hidden layer nodes when the model is initialized. This requirement affects the online learning capability of OS-ELM during the initialization phase. Additionally, OS-ELM treats all training samples equally, and this uniform network weight updating scheme may not accurately capture the relationships between current and previous variables in time series prediction. Furthermore, the input weights of OS-ELM cannot be adjusted, which is usually regarded as a defect of this type of models.

Subsequently, several researchers [27–29] introduced the concept of forgetting factors in OS-ELM. These methods involve selectively updating the output weights while keeping the input weights fixed during the sequence learning phase. In addition, Park et al. [30] proposed a modified version that adjusts input weights through the application of an autoencoder and normalization method. These OS-ELM-based methods have proven effective and have enhanced the performance of time series prediction with OS-ELM. However, Updating only the output weights may affect the model's ability of capturing long-term patterns in time series data, and applying autoencoder to modify OS-ELM can result in a more complex network structure.

To effectively capture the long-term patterns within a lightweight neural network, we introduce in this paper a novel model called RVFL-LSTM which incorporates the long-short term memory mechanism into the simple architecture of RVFL for performing an auto-regression task. It is noted that RVFL-LSTM has the ability to capture both long-term and short-term patterns, which is the key feature that makes our method well-suited for time series prediction. Moreover, for the sake of clarity, this paper will continue using the term "RVFL" instead of "ELM".

The main contributions of this paper are listed below.

(1) This paper develops a lightweight model with long-short term memory for time series, called RVFL-LSTM which has a mechanism of incremental learning with moving auto-regression for online sequence, and its long-short term memory functionality is clearly interpretable.

(2) Different from the traditional RVFL and its many variants, the input weights in the model are adjustable during training, and the adjustment is interpreted as learning the short-term patterns in time series.

(3) A new scheme to update the output weights for RVFL in online sequence learning is given, which is interpreted as the learning of long-term patterns.

(4) The superiority of the model is validated on different real world datasets in comparison with the existing methods of time series prediction.

The rest of this paper is organized as follows. In Section 2, the Auto-Regression (AR), RVFL, and OS-ELM are reviewed. In Section 3, our RVFL-LSTM model is presented and discussed in detail. In Section 4, the experimental analysis on the proposed model and related comparisons are conducted. Section 5 concludes this paper.

2. Backgrounds

2.1. Moving auto-regression (MAR)

Before introducing the concept of MAR, it is important to understand the definition of AR. AR was first proposed by Yule [31] in 1927 as a statistical modeling technique for exploring correlations among observations in a time series. The AR model assumes that future observations can be generated by a linear combination of past observations, and the number of past observations used for prediction is referred to as the lag order (denoted as l). Mathematically, the definition of AR can be represented by Eq. (1):

$$\hat{x}_{l+1} = \varphi_1 x_1 + \varphi_2 x_2 + \cdots + \varphi_l x_l + \varepsilon_{l+1} \tag{1}$$

where $\hat{x}_{l+1}$ represents the predicted value at the moment $l+1$, φ_i is the autoregressive coefficient of the model, and it can be estimated by the least squares procedure or method of moments (through Yule–Walker equations).x_l is the actual observation at time l, and ε represents the white noise error.

In general, the AR model uses the recursive strategy for time series prediction. This paper introduces the direct strategy for time series prediction. Specifically, d observations from the past are used to predict d observations in the future. The direct prediction strategy is one of the main strategies for long-term time series prediction, the detailed description can be seen in Ref. [22]. The direct strategy can be expressed as:

$$\hat{\boldsymbol{x}}_{d+1:2d} = \boldsymbol{\Phi} \cdot \boldsymbol{x}_{1:d} + \boldsymbol{\varepsilon} \tag{2}$$

In Eq. (2), $\boldsymbol{x}_{1:d} = [x_1, x_2, \ldots, x_d]^T$, where x_i represents the past ith observation. $\hat{\boldsymbol{x}}_{d+1:2d} = [\hat{x}_{d+1}, \hat{x}_{d+2}, \ldots, \hat{x}_{2d}]^T$, where x_{d+i} represents the future $(d+i)$-th predicted value. $\boldsymbol{\Phi} \in \mathbb{R}^{d\times d}$ is the coefficient matrix. $\boldsymbol{\varepsilon} \in \mathbb{R}^{d\times 1}$ represents the white noise error. The estimation of $\boldsymbol{\Phi}$ in AR can be achieved through methods like least squares or gradient descent. This paper uses the gradient descent method. The prediction error $\mathcal{L}$ is denoted by Eq. (3):

$$\mathcal{L} = \left\| \hat{\boldsymbol{x}}_{d+1:2d} - \boldsymbol{x}_{d+1:2d} \right\|^2 \tag{3}$$

The gradient of the prediction error is given by Eq. (4):

$$\frac{\partial \mathcal{L}}{\partial \varphi_{i,j}} = \sum_{c=1}^{k} 2(\hat{x}_{d+c} - x_{d+c}) x_j \tag{4}$$

In Eq. (4), $\varphi_{i,j} \in \boldsymbol{\Phi}$, $x_j \in \boldsymbol{x}_{d+1:2d}$, $i, j = 1, 2, \ldots, d$. Next, the parameters $\boldsymbol{\Phi}$ can be updated according to Eq. (5):

$$\varphi_{i,j} = \varphi_{i,j} + r \cdot \frac{\delta \mathcal{L}}{\delta \varphi_{i,j}} \tag{5}$$

In Eq. (5), r represents the learning rate. By iteratively fine-tuning the parameters $\boldsymbol{\Phi}$, the training of the AR model can be accomplished.

Then, MAR can be introduced based AR. In accordance with Eq. (2), the time series data are partitioned into equally sized d-dimensional vectors, as shown below.

Fig. 1. Pretreatment on the original time series.

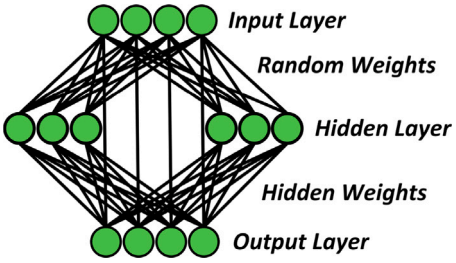


Fig. 2. Random vector functional link.

As shown in Fig. 1, assuming that the number of the d-dimensional vectors is z after preprocessing. MAR aims to partition the data into multiple windows $[U_1, U_2, \ldots]$ and utilizes the sliding window to optimize model parameters within each window. The parameter is updated through the following equation:

$$\varphi_{i,j} = \varphi_{i,j} + r \cdot \frac{\delta \mathcal{L}(U_t)}{\delta \varphi_{i,j}}, t = 1, 2, \ldots, t \leq z \tag{6}$$

The parameters $\boldsymbol{\Phi}$ are continuously updated by sliding the window. Once the training process of MAR is completed, the predictions for the next d time steps can be obtained by inputting the last k observations of the time series (i.e., the last k-dimensional vector after preprocessing).

While it is possible to use the MAR model to predict future observations from $d+1$ to $2d$ time steps by employing the predicted results as input, it is important to note that, in general, doing so may reduce prediction accuracy. This process is straightforward and intuitive, using predicted results as input can accumulate errors and lead to less accurate predictions.

Remark 1. AR is a model that characterizes the relationship between an observation and its previous observations in time series. It assumes that current observation is formed by a linear combination of past observations. MAR can be viewed as a variant of the AR model which introduces the concept of the sliding window. It takes multiple past observations at different time points as inputs and predicts current observations. MAR considers a larger number of past observations, and it is able to capture more complex sequence relationships. However, it is essential to note that the MAR model remains a linear formula.

2.2. Random vector functional link (RVFL)

RVFL is a notable SLFN with random weights, and it was initially proposed by Pao et al. [17] in 1992. In comparison to traditional backpropagation-based neural networks, RVFL boasts a simpler structure and faster training speed. The structure of RVFL is visually depicted in Fig. 2.

The formulations of the RVFL model are introduced. Let $\boldsymbol{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n]^T$ represents the training data, where $\boldsymbol{x}_i = [x_{i1}, \ldots, x_{id}] \in \mathbb{R}^{1\times d}$, $i = 1, 2, \ldots, n$ is the ith training sample. Let $\boldsymbol{Y} = [\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_n]^T$ represents the target matrix, where $\boldsymbol{y}_i = [y_{i1}, \ldots, y_{id}] \in \mathbb{R}^{1\times d}$, $i = 1, 2, \ldots, n$ is the ith corresponding label. Here, n represents the number of training samples. d is the dimension of features and labels. In this paper, the dimension of features is the same as the dimension of the labels. Mathematically, RVFL can be written as

$$f(\boldsymbol{x}_i) = \boldsymbol{\beta}_1 \boldsymbol{x}_i + \boldsymbol{\beta}_2 g(\boldsymbol{W}\boldsymbol{x}_i^T + \boldsymbol{b}) \tag{7}$$

In Eq. (7), $f(\boldsymbol{x}_i)$ represents the output of RVFL, $\boldsymbol{\beta}_1 \in \mathbb{R}^{d\times d}$ is the weight matrix between the input and the output layers, and $\boldsymbol{\beta}_2 \in \mathbb{R}^{d\times N}$ is the weight matrix between the hidden and the output layers. N is the number of nodes in the hidden layer. $\boldsymbol{W} = [\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_N]^T \in \mathbb{R}^{N\times d}$ represents the random weight matrix between the input and the hidden layers, $\boldsymbol{w}_i = [w_{j1}, \ldots, w_{jd}] \in \mathbb{R}^{1\times k}$, $j = 1, 2, \ldots, d$ is the weight vector connecting the jth hidden node and the input nodes. $\boldsymbol{b} = [b_1, \ldots, b_N]^T \in \mathbb{R}^{N\times 1}$ is the bias vector. As mentioned before, $\boldsymbol{W}$ and $\boldsymbol{b}$ are initialized from a uniform interval $[-\lambda, \lambda]$. g represents the activation function. $\boldsymbol{\beta}_1$ and $\boldsymbol{\beta}_2$ can be obtained by the following optimization objective

$$\text{Minimize: } \|\boldsymbol{H}\boldsymbol{\beta} - \boldsymbol{Y}\| \tag{8}$$

In Eq. (8), $\boldsymbol{\beta} = [\boldsymbol{\beta}_1, \boldsymbol{\beta}_2]^T \in \mathbb{R}^{(d+N)\times d}$, $\boldsymbol{H} = [\boldsymbol{H}_1, \boldsymbol{H}_2] \in \mathbb{R}^{n\times(d+N)}$. Here, $\boldsymbol{H}_1$ represents the input data, $\boldsymbol{H}_2$ is the output matrix of the hidden layer.

$$\boldsymbol{H}_1 = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}_{n\times d} \tag{9}$$

$$\boldsymbol{H}_2 = \begin{bmatrix} g(\boldsymbol{w}_1\boldsymbol{x}_1^T + b_1) & \cdots & g(\boldsymbol{w}_N\boldsymbol{x}_1^T + b_N) \\ g(\boldsymbol{w}_1\boldsymbol{x}_2^T + b_1) & \cdots & g(\boldsymbol{w}_N\boldsymbol{x}_2^T + b_N) \\ \vdots & \ddots & \vdots \\ g(\boldsymbol{w}_1\boldsymbol{x}_n^T + b_1) & \cdots & g(\boldsymbol{w}_N\boldsymbol{x}_n^T + b_N) \end{bmatrix}_{n\times N} \tag{10}$$

The solution for Eq. (8) can be expressed as:

$$\boldsymbol{\beta} = \boldsymbol{H}^{\dagger}\boldsymbol{Y} \tag{11}$$

here, $\boldsymbol{H}^{\dagger}$ represents the Moore–Penrose pseudo-inverse [32], and it can be obtained using Eq. (12):

$$\boldsymbol{H}^{\dagger} = \begin{cases} (\boldsymbol{H}^T\boldsymbol{H})^{-1}\boldsymbol{H}^T, & \text{if } \boldsymbol{H}^T\boldsymbol{H} \text{ is nonsingular,} \\ \boldsymbol{H}^T(\boldsymbol{H}\boldsymbol{H}^T)^{-1}, & \text{otherwise.} \end{cases} \tag{12}$$

The above descriptions outline the fundamental form of RVFL. Huang et al. [20] proposed a variant of RVFL where the connections between the input and the output layers are removed. Mathematically, it can be written as:

$$f(\boldsymbol{x}_i) = \boldsymbol{\beta} g(\boldsymbol{W}\boldsymbol{x}_i^T + \boldsymbol{b}) \tag{13}$$

In Eq. (13), $\boldsymbol{\beta}$ represents the weights matrix between the hidden and the output layers. In this newly designed RVFL, $\boldsymbol{\beta}$ is obtained by the following optimization objective.

$$\text{Minimize: } \|\boldsymbol{H}\boldsymbol{\beta} - \boldsymbol{Y}\| \textit{ and } \|\boldsymbol{\beta}\| \tag{14}$$

where $\|\cdot\|$ represents the Frobenius norm. $\boldsymbol{H}$ is the output matrix of the hidden layer.$\boldsymbol{Y}$ is the target matrix. $\|\boldsymbol{H}\boldsymbol{\beta} - \boldsymbol{Y}\|$ represents the error between the ground-truth and the model-computed values, and it is the primary objective to be minimized. $\|\boldsymbol{\beta}\|$ represents the norm of the model parameters which acts as a regularization term for the optimization problem, and it helps to constrain the model complexity and enhance the generalization ability.

The above objective can be transformed into the following form.

$$\begin{aligned} &\min_{\boldsymbol{\beta}} \tfrac{1}{2}\|\boldsymbol{\beta}\|^2 + \tfrac{1}{2}C\|\boldsymbol{\xi}\|^2 \\ &\text{s.t.} \boldsymbol{H}\boldsymbol{\beta} - \boldsymbol{Y} = \boldsymbol{\xi} \end{aligned} \tag{15}$$

where C is a hyperparameter, and it is used to balance the two regularization terms. $\boldsymbol{\xi} = [\xi_1, \xi_2, \ldots, \xi_n]$ is the training error. Using the

method of Lagrange multipliers, Eq. (15) can be transformed into an unconstrained problem.

$$\mathcal{L}(\beta,\xi,\lambda)=\frac{1}{2}\|\beta\|^2+\frac{1}{2}C\|\xi\|^2-\lambda^T(H\beta-Y-\xi) \tag{16}$$

where λ represents the Lagrange multiplier. Based on the Karush–Kuhn–Tucker (KKT) theory, the corresponding KKT conditions are shown as follows:

$$\begin{cases}\frac{\partial\mathcal{L}}{\partial\beta}=0\rightarrow\beta=H^T\lambda\\ \frac{\partial\mathcal{L}}{\partial\xi}=0\rightarrow\lambda=-C\xi\\ \frac{\partial\mathcal{L}}{\partial\lambda}=0\rightarrow H\beta-Y-\xi=O\end{cases} \tag{17}$$

Based on the difference between the number of training samples n and the number of hidden nodes N, the solution for β can be obtained in two different cases. When n is greater than N, λ can be written as $\lambda=-C(H\beta-Y)$, and then, we have

$$\beta=(\frac{I}{C}+H^TH)^{-1}H^TY \tag{18}$$

When n is less than N. The λ can be written as $\lambda=(\frac{I}{C}+H^TH)^{-1}Y$. Then, we have

$$\beta=H^T(\frac{I}{C}+HH^T)^{-1}Y \tag{19}$$

Therefore, β can be shown as:

$$\beta=\begin{cases}(\frac{I}{C}+H^TH)^{-1}H^TY, & n\geq N\\ H^T(\frac{I}{C}+HH^T)^{-1}Y, & n<N\end{cases} \tag{20}$$

Remark 2. Training RVFL does not require a complex parameter tuning process, and RVFL has very high learning convergence speed. However, RVFL cannot learn the sequential data. The new data must be mixed with the original data, and then the mixed data are used to retrained the model. RVFL is not suitable for online sequence learning.

2.3. Online sequential extreme learning machine (OS-ELM)

OS-ELM [26] has been widely used for the sequential data, and applies the recursive least squares method [33] to ELM. The learning performance of the OS-ELM and RVFL is same. OS-ELM contains two different training phases.

The first is the initialization phase. OS-ELM is initialized using the initial training data $X_0=[x_1,x_2,\dots,x_{n_0}]^T$ and the label $Y_0=[y_1,y_2,\dots,y_{n_0}]^T$ from the given training set. Here, the number of training data (n_0) must be greater than or equal to the number of hidden layer nodes (N). β_0 is the output weights, and H_0 is the outputs of the hidden layer. Then we have

$$H_0=\begin{bmatrix}g(w_1x_1^T+b_1) & \cdots & g(w_Nx_1^T+b_N)\\ g(w_1x_2^T+b_1) & \cdots & g(w_Nx_2^T+b_N)\\ \vdots & \ddots & \vdots\\ g(w_1x_{n_0}^T+b_1) & \cdots & g(w_Nx_{n_0}^T+b_N)\end{bmatrix}_{n_0\times N} \tag{21}$$

$$\beta_0=P_0H_0^TY_0 \tag{22}$$

where

$$P_0=(H_0^TH_0)^{-1} \tag{23}$$

The second is the sequential learning phase. The new data $X_{k+1}=[x_1,x_2,\dots,x_{n_{k+1}}]^T$ and the new label $Y_{k+1}=[y_1,y_2,\dots,y_{n_{k+1}}]^T$ are presented. $k+1$ indicate the order of the new data. The n_{k+1} is the number of new data. H_{k+1} is the output matrix of the hidden layer for the newd data. Once the data are used, it will be discarded and not used again in the sequential learning phase. The output weights β_{k+1} becomes

$$\begin{aligned}\beta_{k+1}&=\left(\begin{bmatrix}H_k\\H_{k+1}\end{bmatrix}^T\begin{bmatrix}H_k\\H_{k+1}\end{bmatrix}\right)^{-1}\begin{bmatrix}H_k\\H_{k+1}\end{bmatrix}^T\begin{bmatrix}Y_k\\Y_{k+1}\end{bmatrix}\\&=\left(H_k^TH_k+H_{k+1}^TH_{k+1}\right)^{-1}\left(H_k^TY_k+Y_{k+1}^TH_{k+1}\right)\end{aligned} \tag{24}$$

Then, the β_{k+1} can be written as (the specific derivation process can be found in Ref. [26].)

$$\beta_{k+1}=\beta_k+P_{k+1}H_{k+1}^T\left(Y_{k+1}-H_{k+1}^T\beta_k\right) \tag{25}$$

$$P_{k+1}=P_k-P_kH_{k+1}^T\left(I+H_{k+1}P_kH_{k+1}^T\right)^{-1}H_{k+1}^TP_k \tag{26}$$

When new data continues to be received, P_{k+1} can be continuously updated.

Remark 3. It is essential n_0 should not be less than N in OS-ELM. And the n_{k+1} is not constant. When $n_{k+1}=1$, Eq. (25) and (26) can be written as

$$\beta_{k+1}=\beta_k+P_{k+1}h_{k+1}^T\left(y_{k+1}^T-h_{k+1}^T\beta_k\right) \tag{27}$$

$$P_{k+1}=P_k-\frac{P_kh_{k+1}h_{k+1}^TP_k}{1+h_{k+1}^TP_kh_{k+1}} \tag{28}$$

In Eq. (27), $h_{k+1}=[g(w_1x_{n_{k+1}}^T+b_1),\dots,g(w_Nx_{n_{k+1}}^T+b_N)]$. Besides, the input weights of OS-ELM cannot be adjusted when the initialization phase is completed. OS-ELM treats all training samples equally and adopts the recursive updating mode for β_k. This learning mode may not be entirely suitable for time series prediction.

3. Random vector functional link with long-short term memory (RVFL-LSTM)

This section provides the details of RVFL-LSTM, which pays particular attention to explaining the updating of input weights in RVFL-LSTM and deriving the method for updating output weights during the process of sliding windows. Additionally, we explain how the proposed model captures both the short-term and long-term patterns in time series prediction.

3.1. The proposed RVFL-LSTM method

RVFL-LSTM is a SLFN with no direct connections between the input and output layers. It incorporates the concept of MAR into RVFL. During the training process, the input weights are adjusted by the BP algorithm and the output weights are updated. RVFL-LSTM employs the Direct strategy for long-term prediction of time series, i.e., the past d observations are used to predict the future d observations. The overview of RVFL-LSTM is given in Fig. 3.

Given a specific time series $s=[s_1,s_2,\dots,s_T]^T$. When the data preprocessing is completed, we can obtain

$$\begin{aligned}X&=[x_1,x_2,\dots,x_{z-1}]^T\\ Y&=[x_2,x_3,\dots,x_z]^T\end{aligned} \tag{29}$$

where $X\in\mathbb{R}^{(z-1)\times d}$ represents the training data, $Y\in\mathbb{R}^{(z-1)\times d}$ is the corresponding label, and $T=z\times d$. z is the number of the features after data preprocessing. d is the dimension of the features. $x_i=[s_{(i-1)d+1},s_{(i-1)d+2},\dots,s_{id}]\in\mathbb{R}^{1\times d}, i=1,2,\dots,z-1$ represents the ith dimension in X (the $((i-1)d+1)$-th observation to ik-th observation in the time series s).

RVFL-LSTM contains two training phases.

Phase 1: The RVFL-LSTM network is initialized where the numbers of nodes for input, output, and hidden layers are set to be d, d, and N respectively. The random weights W_0 and the biases b_0 are initialized randomly from $[-\lambda,\lambda]$, respectively. RVFL-LSTM adopts the sliding

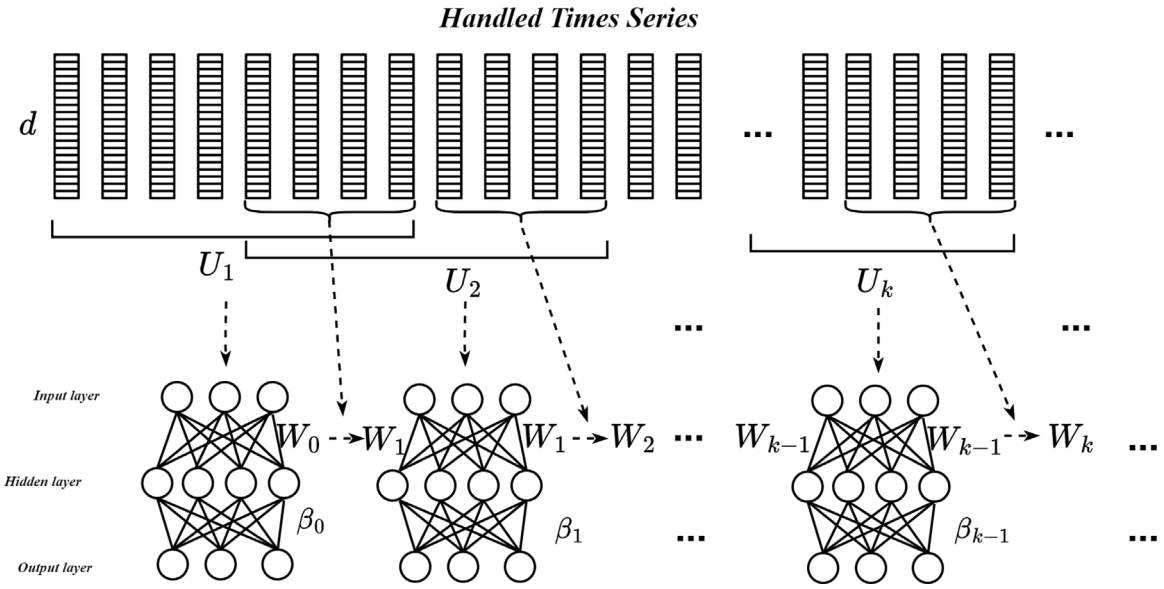


Fig. 3. Random vector functional link with long-short term memory.

window mechanism for the updating of parameters. The initial window can be formulated as follows:

$$\begin{aligned} \boldsymbol{U}_x^{(1)} &= [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_u]^T \\ \boldsymbol{U}_y^{(1)} &= [\boldsymbol{x}_2, \boldsymbol{x}_3, \ldots, \boldsymbol{x}_{u+1}]^T \end{aligned} \tag{30}$$

where u is the window size. $\boldsymbol{U}_x^{(1)}$ is the training data in the initialization window. $\boldsymbol{U}_y^{(1)}$ is the corresponding label.

The output $\boldsymbol{H}_0$ of the hidden layer is :

$$\boldsymbol{H}_0 = g(\boldsymbol{W}_0(\boldsymbol{U}_x^{(1)})^T + \boldsymbol{b}_0) \tag{31}$$

Based on the Eq. (20), we can initialize the output weights $\boldsymbol{\beta}_0$ as:

$$\boldsymbol{\beta}_0 = \begin{cases} (\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_0{}^T\boldsymbol{H}_0)^{-1}\boldsymbol{H}_0^T\boldsymbol{U}_y^{(1)}, & u \geq N \\ \boldsymbol{H}_0^T(\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_0\boldsymbol{H}_0^T)^{-1}\boldsymbol{U}_y^{(1)}, & u < N \end{cases} \tag{32}$$

One of the essential points about RVFL-LSTM is the following. We consider that the part of the window closer to the new arrival observations should have a greater weight (i.e., the short-term patterns in time series).

Next, RVFL-LSTM uses a batch of latter observations in the window to optimize $\boldsymbol{W}_0$ and $\boldsymbol{b}_0$ based on the BP algorithm, which is shown as:

$$\begin{aligned} \boldsymbol{W}_1 &= \boldsymbol{W}_0 + r \cdot \frac{\partial \mathcal{L}_1}{\partial \boldsymbol{W}_0} \\ \boldsymbol{b}_1 &= \boldsymbol{b}_0 + r \cdot \frac{\partial \mathcal{L}_1}{\partial \boldsymbol{b}_0} \\ \mathcal{L}_1 &= \left\| \boldsymbol{U}_y^{(1)} \odot \boldsymbol{M} - g\left(\boldsymbol{W}_0(\boldsymbol{U}_x^{(1)} \odot \boldsymbol{M})^T + \boldsymbol{b}_0\right)\boldsymbol{\beta}_0 \right\|^2 \end{aligned} \tag{33}$$

where $\boldsymbol{W}_1$ and $\boldsymbol{b}_1$ are the updated parameters. r is the learning rate. $\mathcal{L}_1$ represents the prediction error of $\boldsymbol{U}_x^{(1)} \odot \boldsymbol{M}$. $\odot$ represents the Hadamard product. $\boldsymbol{M}$ is a matrix and has the same dimension as $\boldsymbol{U}_x^{(1)}$. The last m rows of elements in $\boldsymbol{M}$ are set to 1, and the rest are set to 0. The details of optimizing random weights by the BP algorithm will be introduced in Section 3.2.

Phase 2: The window starts sliding backward, and the new training samples are introduced. The new window is formulated as

$$\begin{aligned} \boldsymbol{U}_x^{(2)} &= [\boldsymbol{x}_{1+L}, \boldsymbol{x}_{2+L}, \ldots \boldsymbol{x}_{u+L}]^T \\ \boldsymbol{U}_y^{(2)} &= [\boldsymbol{x}_{2+L}, \boldsymbol{x}_{3+L}, \ldots \boldsymbol{x}_{u+L+1}]^T \end{aligned} \tag{34}$$

where L is the step size of the window sliding. The output of the hidden layer becomes:

$$\boldsymbol{H}_1 = g(\boldsymbol{W}_1(\boldsymbol{U}_x^{(2)})^T + \boldsymbol{b}_1) \tag{35}$$

Then, $\boldsymbol{\beta}_1$ is obtain based on $\boldsymbol{H}_1$, which is realized through the following optimization objective

$$\text{Minimize: } \left\| \boldsymbol{H}_1\boldsymbol{\beta}_1 - \boldsymbol{U}_y^{(2)} \right\| \textit{ and } \left\| \boldsymbol{\beta}_1 - \boldsymbol{\beta}_0 \right\| \tag{36}$$

where $\left\| \boldsymbol{H}_1\boldsymbol{\beta}_1 - \boldsymbol{U}_y^{(2)} \right\|$ represents the training error obtained based on the samples in the new window. By minimizing the term, the model can fit the observations in the new window. $\left\| \boldsymbol{\beta}_1 - \boldsymbol{\beta}_0 \right\|$ represents the distance between original $\boldsymbol{\beta}_0$ and updated $\boldsymbol{\beta}_1$. By minimizing this term, the retention of the previously trained model can be achieved.

Based on KKT theory and the method of Lagrange multiplier, we have

$$\boldsymbol{\beta}_1 = \begin{cases} \left(\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_1^T\boldsymbol{H}_1\right)^{-1}\boldsymbol{H}_1^T(\boldsymbol{U}_y^{(2)} - \boldsymbol{H}_1 \cdot \boldsymbol{\beta}_0) + \boldsymbol{\beta}_0, & u \geq N \\ \boldsymbol{H}_1^T\left(\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_1\boldsymbol{H}_1^T\right)^{-1}(\boldsymbol{U}_y^{(2)} - \boldsymbol{H}_1 \cdot \boldsymbol{\beta}_0) + \boldsymbol{\beta}_0, & u < N \end{cases} \tag{37}$$

Details about the derivation of $\boldsymbol{\beta}_1$ are shown in Section 3.1. Then $\boldsymbol{W}_2$ and $\boldsymbol{b}_2$ can be obtained based on the $\boldsymbol{W}_1$ and $\boldsymbol{b}_1$ using the Eq. (33).

$$\begin{aligned} \boldsymbol{W}_2 &= \boldsymbol{W}_1 + r \cdot \frac{\partial \mathcal{L}_2}{\partial \boldsymbol{W}_1} \\ \boldsymbol{b}_2 &= \boldsymbol{b}_1 + r \cdot \frac{\partial \mathcal{L}_2}{\partial \boldsymbol{b}_1} \\ \mathcal{L}_2 &= \left\| \boldsymbol{U}_y^{(2)} \odot \boldsymbol{M} - g\left(\boldsymbol{W}_1(\boldsymbol{U}_x^{(2)} \odot \boldsymbol{M})^T + \boldsymbol{b}_1\right)\boldsymbol{\beta}_1 \right\|^2 \end{aligned} \tag{38}$$

The above process is the initialization phase and one iteration of window sliding in RVFL-LSTM. As new observations are introduced with the continued sliding of the window, the parameters of RVFL-LSTM can be continuously updated using Eqs. (37) and (38). The proposed RVFL-LSTM method is detailed in Algorithm 1.

Remark 4. In time series prediction, there is a common belief that the newly arrived observations will have a greater impact on the predictions. The new observations are usually considered to contain recent information, and this information can be referred to as the short-term patterns of a time series. Long-term patterns in a time series are also crucial, they represent more enduring trends, seasonality, periodicity, or other long-term variations. Considering both the short-term and long-term patterns is effective because it can capture changes and influences at different time scales in a time series. The rational combination of these two aspects often leads to improved prediction accuracy. RVFL-LSTM uses the $\boldsymbol{W}$ to capture the short-term patterns and uses the $\boldsymbol{\beta}$ to capture the long-term patterns in time series.

3.2. Updating output weights β via a regularization

During the window sliding process, it becomes necessary to update the weights $\boldsymbol{\beta}$ in the RVFL-LSTM model. These updated weights $\boldsymbol{\beta}$ are instrumental in capturing and retaining valuable information from the newly introduced observations. In this subsection, we present a comprehensive explanation of the procedure for updating $\boldsymbol{\beta}$ within the context of the RVFL-LSTM sliding window mechanism.

The window slides is assumed that from $\boldsymbol{U}_k$ to $\boldsymbol{U}_{k+1}$. k represents the order of the sliding windows. Based on the Eq. (36), we have the optimization objective

$$\text{Minimize: } \left\|\boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)}\right\| \text{ and } \left\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\right\| \tag{39}$$

where $\boldsymbol{H}_k = g(\boldsymbol{W}_k(\boldsymbol{U}_x^{(k+1)})^T + \boldsymbol{b}_k)$, $\boldsymbol{\beta}_k$ is the updated weights based on the observations in $\boldsymbol{U}_{k+1}$. $\boldsymbol{U}_y^{(k+1)}$ is the labels of training data in $\boldsymbol{U}_{k+1}$. $\boldsymbol{\beta}_{k-1}$ is the weights from the last model. Eq. (39) serves as the optimization objective in our approach. This optimization objective ensures that the updated $\boldsymbol{\beta}_k$ can effectively adapt to the new window observations while preserving pertinent information from the previous model. Subsequently, we formulate the following optimization problem.

$$\begin{aligned} &\min_{\boldsymbol{\beta}_k} \tfrac{1}{2}\left\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\right\|^2 + \tfrac{1}{2}C\left\|\boldsymbol{\xi}_{k+1}\right\|^2 \\ &\text{s.t.} \boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)} = \boldsymbol{\xi}_{k+1} \end{aligned} \tag{40}$$

where $\boldsymbol{\xi}_{k+1}$ represents the training error, C is a hyperparameter. Then, we transform it into an unconstrained problem.

$$\begin{aligned} \mathcal{L}(\boldsymbol{\beta}_k, \boldsymbol{\xi}_{k+1}, \lambda) = &\frac{1}{2}\left\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\right\|^2 + \frac{1}{2}C\left\|\boldsymbol{\xi}_{k+1}\right\|^2 \\ &-\lambda^T(\boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)} - \boldsymbol{\xi}_{k+1}) \end{aligned} \tag{41}$$

where λ represents the Lagrange multiplier. The KKT conditions for Eq. (41) are shown as follows

$$\begin{cases} a: \dfrac{\partial\mathcal{L}}{\partial\boldsymbol{\beta}_k} = 0 \rightarrow\rightarrow \boldsymbol{\beta}_k = \boldsymbol{H}_k^T\lambda + \boldsymbol{\beta}_{k-1} \\ b: \dfrac{\partial\mathcal{L}}{\partial\boldsymbol{\xi}_{k+1}} = 0 \rightarrow\rightarrow \lambda = -C\boldsymbol{\xi}_{k+1} \\ c: \dfrac{\partial\mathcal{L}}{\partial\lambda} = 0 \rightarrow\rightarrow \boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)} - \boldsymbol{\xi}_{k+1} = \boldsymbol{O} \end{cases} \tag{42}$$

We would solve $\boldsymbol{\beta}_k$ in two different cases. The first is that the window size u is smaller than N. According to Eq. ((42)b) and Eq. ((42)c), we have

$$\boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)} = -\frac{\boldsymbol{I}}{C}\lambda \tag{43}$$

then, based the Eq. ((42)a), we have

$$\lambda = \left(\boldsymbol{H}_k\boldsymbol{H}_k^T + \frac{\boldsymbol{I}}{C}\right)^{-1}\left(\boldsymbol{U}_y^{k+1} - \boldsymbol{H}_k\boldsymbol{\beta}_{k-1}\right) \tag{44}$$

and the $\boldsymbol{\beta}_k$ is

$$\boldsymbol{\beta}_k = \boldsymbol{H}_k^T\left(\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_k\boldsymbol{H}_k^T\right)^{-1}\left(\boldsymbol{U}_y^{(k+1)} - \boldsymbol{H}_k\boldsymbol{\beta}_{k-1}\right) + \boldsymbol{\beta}_{k-1} \tag{45}$$

The second case is that the window size u is greater than N. we can obtain

$$\lambda = -C\left(\boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)}\right) \tag{46}$$

By combining the Eq. (44) and Eq. ((42)a), the $\boldsymbol{\beta}_k$ can be obtain as follow.

$$\begin{aligned} \boldsymbol{\beta}_k &= -C\boldsymbol{H}_k\left(\boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^k\right) + \boldsymbol{\beta}_{k-1} \\ &= \left(\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_k^T\boldsymbol{H}_k\right)^{-1}\left(\boldsymbol{H}_k^T\boldsymbol{U}_y^{(k+1)} + \frac{\boldsymbol{I}}{C}\boldsymbol{\beta}_{k-1}\right) \\ &= \left(\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_k^T\boldsymbol{H}_k\right)^{-1}\boldsymbol{H}_k^T\left(\boldsymbol{U}_y^{(k+1)} - \boldsymbol{H}_k\boldsymbol{\beta}_{k-1}\right) + \boldsymbol{\beta}_{k-1} \end{aligned} \tag{47}$$

Thus, we can derive a more comprehensive form for Eq. (37), which is shown in Eq. (48).

$$\boldsymbol{\beta}_k = \begin{cases} \left(\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_k^T\boldsymbol{H}_k\right)^{-1}\boldsymbol{H}_k^T\left(\boldsymbol{U}_y^{(k+1)} - \boldsymbol{H}_k\boldsymbol{\beta}_{k-1}\right) + \boldsymbol{\beta}_{k-1}, & u \geq N \\ \boldsymbol{H}_k^T\left(\frac{\boldsymbol{I}}{C} + \boldsymbol{H}_k\boldsymbol{H}_k^T\right)^{-1}\left(\boldsymbol{U}_y^{(k+1)} - \boldsymbol{H}_k\boldsymbol{\beta}_{k-1}\right) + \boldsymbol{\beta}_{k-1}, & u < N \end{cases} \tag{48}$$

Remark 5. We derive a novel mechanism for updating $\boldsymbol{\beta}$ in RVFL-LSTM during online sequence learning. This mechanism enhances training efficiency without the need to retrain the entire RVFL with all observations during the sliding window process. It ensures that the model can effectively capture long-term patterns. Furthermore, in contrast to other online sequential learning methods for RVFL, the derived mechanism has no constraint during the initialization phase, i.e., the training can be regularly conducted no matter whether the number of training data is greater or less than the number of hidden layer nodes in the network.

3.3. Updating random weights via BP algorithm

This subsection provides a detailed explanation of using the BP algorithm to update the model parameters $\boldsymbol{W}$ and $\boldsymbol{b}$. After each adjustment of the hidden weights $\boldsymbol{\beta}$ in RVFL-LSTM, the model parameters $\boldsymbol{W}$ and $\boldsymbol{b}$ are optimized by using the BP algorithm. We adjust the $\boldsymbol{W}$ and $\boldsymbol{b}$ using the m last rows of $\boldsymbol{U}_x$. The purpose is to make the adjusted $\boldsymbol{W}$ and $\boldsymbol{b}$ fit the later observations better.

Let us assume that the current window is $\boldsymbol{U}_k$, and the training data and corresponding labels in the window are represented as $\boldsymbol{U}_x^{(k)}$ and $\boldsymbol{U}_y^{(k)}$, respectively. The $\boldsymbol{M}$ in Eqs. (33) and (38) can be represented as

$$\boldsymbol{M} = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix}_{u\times d} \tag{49}$$

where u is the window size, the last m rows of elements in $\boldsymbol{M}$ are 1. Here, m can be considered as a hyperparameter, and we can determine how many the observations are used to update $\boldsymbol{W}$ and $\boldsymbol{b}$ by $(\boldsymbol{U}_x^{(k)} \odot \boldsymbol{M})$.

The loss function is

$$\mathcal{L}_k = \left\|\boldsymbol{U}_y^{(k)} \odot \boldsymbol{M} - g\left(\boldsymbol{W}_{k-1}(\boldsymbol{U}_x^{(k)} \odot \boldsymbol{M})^T + \boldsymbol{b}_{k-1}\right)\boldsymbol{\beta}_{k-1}\right\|^2 \tag{50}$$

where $\mathcal{L}_k$ is the prediction error of the $\boldsymbol{U}_x^{(k)} \odot \boldsymbol{M}$ rather than the $\boldsymbol{U}_x^{(k)}$. Then, the $\boldsymbol{W}_k$ and $\boldsymbol{b}_k$ can be updated through

$$\begin{aligned} \boldsymbol{W}_k &= \boldsymbol{W}_{k-1} + r \cdot \frac{\partial\mathcal{L}_k}{\partial\boldsymbol{W}_{k-1}} \\ \boldsymbol{b}_k &= \boldsymbol{b}_{k-1} + r \cdot \frac{\partial\mathcal{L}_k}{\partial\boldsymbol{b}_{k-1}} \end{aligned} \tag{51}$$

Remark 6. Short-term patterns refer to instantaneous or brief changes in a time series, and these patterns typically associate with the most recent observations. In our proposed method, the BP algorithm is employed to adjust the $\boldsymbol{W}$ and $\boldsymbol{b}$, this mechanism enables RVFL-LSTM to capture short-term patterns in time series prediction. The adjusted $\boldsymbol{W}$ and $\boldsymbol{b}$ can provide a better fitting for the recent observations. Generally, when the network size is fixed, the capacity of the neural network is determined. So, using the recent observations as inputs to adjust the model parameters based on the BP algorithm will lead to a phenomenon that the fitting ability of model to the observations beyond the last m ones may decrease. We call this phenomenon as forgetting. This forgetting process primarily assists RVFL-LSTM in capturing short-term patterns.

3.4. Long-term patterns

The long-term patterns are essential for the analysis and prediction of time series data. Capturing these patterns can help us to enhance

the comprehension of the underlying regularities within the time series data and contribute to the prediction performance.

In general, long-term patterns denote the long-range dependency between observations in a time series. In mathematical statistics, a usual definition of long memory for the stationary time series $\boldsymbol{S}_t, t \in \mathbb{Z}$ is that [34]

$$\sum_{l=-\infty}^{\infty} \gamma_S(l) = \infty \tag{52}$$

where $\gamma_S(l), l \in \mathbb{Z}$ is the autocovariance function of $\boldsymbol{S}_t$. $\mathbb{Z}$ represents the set of all integers. $\gamma_S(l)$ can represent the degree of correlation between the variable and itself at a lag l within the time series. Specifically, when the lag order $l \to \infty$ or $l \to -\infty$, $\gamma_S(l) \neq 0$, indicating thta there is still a non-negligible impact. In this paper, we refer to the long-range dependency as the long-term patterns.

In LSTM, the long-term patterns in time series are captured by the complex gating mechanism, and a memory cell is used to store them. In our proposed model, we use $\boldsymbol{\beta}$ to store the long-term patterns in time series. For the specific moving auto-regression task in time series, a regularization term $\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\|$ is set in RVFL-LSTM, where the $\boldsymbol{\beta}_{k-1}$ approximately memorizes the patterns of all observations before the window $\boldsymbol{U}_{k+1}$, and $\boldsymbol{\beta}_k$ is updated based on the $\boldsymbol{\beta}_{k-1}$ and the patterns of the observations in $\boldsymbol{U}_{k+1}$. Specifically, the initialization phase ($k = 0$) in RVFL-LSTM, $\boldsymbol{\beta}_0$ is calculated using the Eq. (32), and the patterns of the observations in the window $\boldsymbol{U}_1$ are captured in $\boldsymbol{\beta}_0$. In the process of window sliding, the regularization term $\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\|$ is minimized to ensure that $\boldsymbol{\beta}_k$ can inherit the patterns from $\boldsymbol{\beta}_{k-1}$ to some extent. During the process of the window sliding backward, it is easy to know that $\boldsymbol{\beta}_k$ can be recursively updated starting from $\boldsymbol{\beta}_0$, and it means that $\boldsymbol{\beta}_k$ contains the patterns of all observations in $\{\boldsymbol{U}_1, \ldots, \boldsymbol{U}_{k+1}\}$ to some extent. When the regularization term $\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\|$ attains its minimum, i.e., $\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\| = 0$, $\boldsymbol{\beta}_k$ can completely inherit the patterns from $\boldsymbol{\beta}_{k-1}$, i.e., $\boldsymbol{\beta}_k = \boldsymbol{\beta}_{k-1}$. However, RVFL-LSTM has two minimization objectives: $\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\|$ and $\left\|\boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)}\right\|$, $\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\|$ is used to ensure that the model inherits the previous patterns during the window sliding process; $\left\|\boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)}\right\|$ ensures that the model fits the observations in $\boldsymbol{U}_{k+1}$. By balancing these two objectives, the inheriting from $\boldsymbol{\beta}_{k-1}$ and the fitting of the observations in $\boldsymbol{U}_{k+1}$ for the model can be achieved simultaneously.

For a stationary time series, when $\gamma_S(l)$ remains non-zero and non-negligible for a long enough period of time, the long memory exists. In RVFL-LSTM, we do not have a mathematical formulation for the long memory. It is important to note that the concept of long-term patterns we use here is not entirely consistent with the long memory defined in classicalstatistics. Therefore, we prefer to use the concept the long-term patterns rather than the long memory. We capture these long-term patterns through minimizing the regularization term $\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\|$. Experimentally, we observe that for a relatively long time series data, our model still provides a good fit for the observations (A detailed description can be found in Section 4.4.). This verifies that our model can capture such long-term patterns in the time series.

LSTM captures the long-term patterns through the gating mechanism and stores them in the memory cell. During the training of LSTM, the model parameters are updated and adjusted through the BP algorithm. However, LSTM does not explicitly give the inheritance relationship for long-term patterns, i.e., LSTM focuses only on the mapping from input to output and does not consider capturing the long-term patterns as its optimization objective. The black-box learning approach of LSTM is hard to interpret. In contrast, our model gives the regularization term $\|\boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1}\|$ to hold long-term patterns and $\left\|\boldsymbol{H}_k\boldsymbol{\beta}_k - \boldsymbol{U}_y^{(k+1)}\right\|$ to ensure the accurate mapping from input to output.

3.5. Short-term patterns

Statistically, the short memory for a stationary time series $\boldsymbol{S}_t, t \in \mathbb{Z}$ is that [34]

$$0 < \sum_{l=-\infty}^{\infty} \gamma_S(l) < \infty \tag{53}$$

The short memory can be understood as $\gamma_S(l) = 0$ or the role of $\gamma_S(l)$ for a time series can be ignored if l is becoming sufficiently large. In other words, the variable of the time series has a short-term impact on the model prediction (in contrast to the long memory). In this paper, we call this short time influence the short-term patterns.

In LSTM, short-term patterns can be stated as the fact that the output of the previous time point serves as one of the inputs for the next moment. However, combined with the definition of short memory in a stationary time series, we believe that these short-term patterns may persist for a period of time (i.e., these short-term patterns involve multiple input–output pairs of the observations). In RVFL-LSTM, these short-term patterns are that, the fitting ability of the trained model is better for the more recently data during the sequential learning process. The short term patterns are obtained by adjusting the input weights $\boldsymbol{W}$. In order to capture these short-term patterns, we use the BP algorithm to adjust the input weights $\boldsymbol{W}$. Specifically, when the sliding window $\boldsymbol{U}_k$ arrives, $\boldsymbol{\beta}_{k-1}$ is firstly updated based on $\boldsymbol{\beta}_{k-2}$, the input weights are $\boldsymbol{W}_{k-1}$ at the moment; then the training loss $\mathcal{L}_k$ is calculated by using the Eq. (50), and $\boldsymbol{W}_k$ is obtained based on $\boldsymbol{W}_{k-1}$. In general, by calculating the training loss and adjusting the model parameters using BP algorithm, it is possible to reduce the training loss of the model and make better fitting for the training data. For the moving auto-regression task, the sliding windows actually overlap each other; the window size is u in the experiments, and the sliding step size is $u/2$. $\mathcal{L}$ is calculated by using the last $u/2$ rows of observations in the window and used to update the $\boldsymbol{W}$. In other words, the adjusted $\boldsymbol{W}$ is more suitable for the more recently arrived observations in the window. Besides, the adjustment occurs only once, and the setting of m is empirical in our experiments. A more straightforward explanation of the short-term patterns in RVFL-LSTM is as follows.

On a given data set A, we can train an RVFL model. When a new data set B is coming, we need to adjust parameters $\boldsymbol{\beta}$ and $\boldsymbol{W}$ of the model RVFL. The adjustment of $\boldsymbol{\beta}$ and $\boldsymbol{W}$ is completed through 2 phases. The first phase is to adjust the weights $\boldsymbol{\beta}$ between hidden and output layers, aiming at the error of the model (on $A \cup B$) attains minimum. In this sense, we can think these parameters (adjusted analytic weights $\boldsymbol{\beta}$) have more importance of all collected data A and B (long term). The second phase is to refine the parameters $\boldsymbol{W}$ between input and hidden layers, which are randomly assigned in the first phase, aiming at the model error to be further reduced particularly on B. Since B is the new data, in this sense, we can think these parameters (changed random weights$\boldsymbol{W}$) have more importance of the new data (short term).

As shown in Fig. 4, the given data set A are the first 24 data points, and the new data set B are the last 24 data points. After the model parameters $\boldsymbol{\beta}$ are adjusted in the first phase, the model minimizes the error (on $A \cup B$). The second phase is to refine the parameters $\boldsymbol{W}$, and the model minimizes the error on B. It can be observed that the fitting ability of the model for the last 24 data points is better than the first 24 data points in the second phase.

In RVFL-LSTM, the idea of capturing the short-term patterns in time series is obviously different from LSTM. The recurrent connection is set in LSTM to capture short-term patterns by serving the output of the previous time point as one of the inputs for the next moment and inevitably leads to the complex network structure. In contrast, RVFL-LSTM captures short-term patterns by adjusting $\boldsymbol{W}$ and has a more lightweight network structure.

Additionally, it is important to note that, although this adjustment ensures better fitting for new observations in the sliding window, the fitting performance on the data outside the new observations may

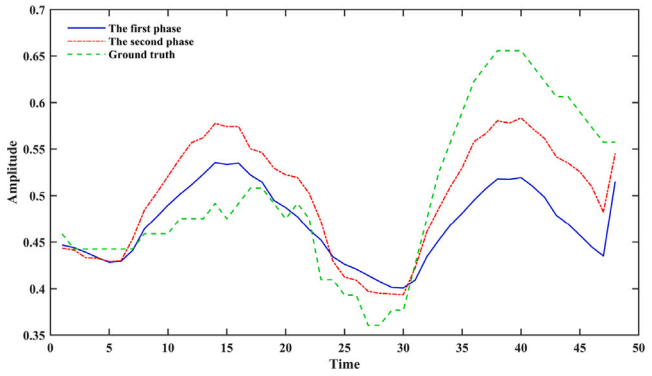


Fig. 4. The short-term patterns of RVFL-LSTM.

decrease to some extent with a certain network capacity (As shown in Fig. 4, the model experiences a degradation in performance on the first 24 data points in the second phase.). However, the decrease does not negatively affect the prediction performance and may improve it on the contrary. Further discussion and analysis are provided in Section 4.4.

Algorithm 1 RVFL-LSTM

Input: Training data $\boldsymbol{X} \in \mathbb{R}^{(z-1)\times d}$ and labels $\boldsymbol{Y} \in \mathbb{R}^{(z-1)\times d}$, the number of hidden layer nodes N, the window size u, the step length of window sliding L, the hyperparameter C, the learning rate r and the mask vector $\boldsymbol{M}$;

Stage 1: Starting with the first sample in $\boldsymbol{X}$, initializing the window and model parameters in RVFL-LSTM.

Randomly initializing $\boldsymbol{W}_0$ and $\boldsymbol{b}_0$ from the interval $[-1, 1]$;
Initializing the window $\boldsymbol{U}_1$;
Calculating initial $\boldsymbol{\beta}_0$ based on Eq. (32);
Masking former observations in the window based on $\boldsymbol{M}$;
Optimizing $\boldsymbol{W}_1$ and $\boldsymbol{b}_1$ based on Eq. (50) and Eq. (51);

Stage 2: Constantly sliding back the window with step length L and adjusting the model parameters based on the new window data.

While the end of window does not reach the end of $\boldsymbol{X}$ and $\mathcal{L}_k$ does not decrease to the threshold:
  Calculating the output of hidden layer $\boldsymbol{H}_k$ based on the new window $\boldsymbol{U}^{k+1}$;
  Optimizing hidden weights $\boldsymbol{\beta}_k$ based on Eq. (48);
  Masking former observations in the window based on $\boldsymbol{M}$;
  Optimizing $\boldsymbol{W}_{k+1}$ and $\boldsymbol{b}_{k+1}$ based on Eq. (50) and Eq. (51);
Return $\boldsymbol{W}_{k+1}$, $\boldsymbol{b}_{k+1}$ and $\boldsymbol{\beta}_k$.

4. Experiments

In this section, we validate our proposed RVFL-LSTM method on real-world datasets. It is worth noting that numerous methods based on OS-ELM or LSTM are available for time series prediction in the references. However, in practical applications, the classical LSTM [12] and OS-ELM [26] are the most widely used methods. In addition, adaptive exponential smoothing (AES) is a classical time series prediction method, but our previous experimental results and the related Refs. [35] show that the performance of the AES method is not as well as the neural network-based method in this paper. Therefore, our work focuses on comparing RVFL-LSTM with OS-ELM and LSTM.

4.1. Description of datasets

We extensively perform experiments on 10 widely used, public benchmark datasets that cover six mainstream time series prediction applications (Electricity, Traffic, Economics, Energy, Weather, and Medicine). All datasets used in our empirical studies are publicly available. Table 1 summarizes the corpus statistics of original datasets. In this paper, we explore the task of univariate time series prediction, which is an autoregressive form. All the data are min–max normalized in our experiments.

Table 1
Dataset Statistics, where T is length of time series, D is number of variables, F is sampling interval of time points.

Dataset	T	D	F	Domain
ETTh1	17 420	7	1 h	Electricity
ETTm1	69 680	7	15 min	Electricity
Traffic	17 544	862	1 h	Traffic
Exchange	7588	8	1 day	Economy
Solar	52 560	137	10 min	Energy
ECL	26 304	321	1 h	Electricity
USLC	43 341	12	1 h	Weather
WSB	52 695	21	10 min	Weather
AirQuality	9358	13	1 h	Weather
ILI	1044	7	1 week	Medicine

Electricity Transformer Temperature [36]: The ETT dataset includes oil temperature and power load features of the electricity transformer temperature from July 2016 to July 2018. ETT dataset comprises hourly-level datasets (ETTh) and 15-minute-level datasets (ETTm).

Traffic [37]: The Traffic dataset describes the hourly road occupancy rates measured by 862 sensors on San Francisco Bay area freeways from January 2015 to December 2016.

Exchange-Rate [37]:The Exchange dataset is the collection of daily exchange rates of eight countries including Australia, British, Canada, Switzerland, China, Japan, New Zealand and Singapore ranging from 1990 to 2016.

Solar-Energye [37]: The Solar dataset records the solar power production in the year of 2006. It is sampled every 10 min from 137 PV plants in Alabama State.

Electricity Consuming Load [37]: The ECL dataset reports the 15 min electricity consumption (KWh) of 321 customers from 2012 to 2014. The data is converted to reflect hourly consumption.

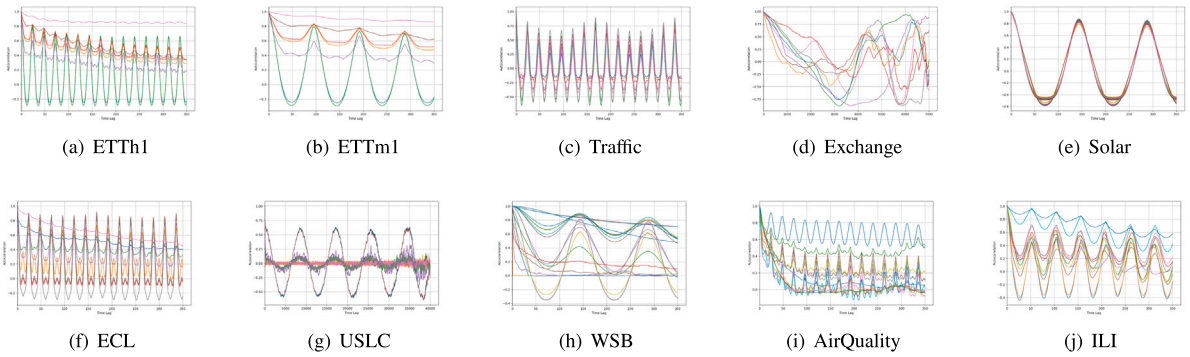
U.S. Local Climatological [36]: The USLC dataset contains local climatological data for nearly 1,600 U.S. locations. Its data includes 12 indicators and is recorded every 1 h. In our experiment, we choose the STATION with number 01001099999 from 2015 to 2019.

WS Beutenberg [38]: The WSB dataset includes meteorological time series from the Weather Station of the Max Planck Biogeochemistry Institute in 2020. It records 21 weather indicators, which are collected every 10 min.

AirQualityUCI [39]: The AirQuality dataset contains 9,358 instances of hourly averaged air quality measurements taken by chemical sensors in an Italian city over the course of one year. It includes 13 indicators of weather.

Influenza-like Illness [38]: The ILI dataset describes the weekly record influenza-like illness patients from Centers for Disease Control and Prevention of the United States from 2002 to 2022. It the ratio of patients seen with influenza-like illness and the number of patients.

We can see in Fig. 5. There are repetitive patterns with high autocovariance on the ETTh1, ETTm1, Traffic, Solar, ECL, Weather-A, Weather-B, AirQuality and ILI datasets. And the short- and the long-term patterns are observable, which reflect the expected regularity on these datasets. We hardly observe any repetitive patterns, expect some short-term local continuity on Exchange dataset. Therefore, the methods which can properly model and successfully leverage both short-term and long-term patterns should outperform well in these datasets.



(a) ETTh1 (b) ETTm1 (c) Traffic (d) Exchange (e) Solar

(f) ECL (g) USLC (h) WSB (i) AirQuality (j) ILI

Fig. 5. The autocorrelation graphs of sampled variables form 10 datasets.

Table 2
The model parameters on different datasets.

Dataset	Number of nodes in input/output layer	Number of nodes in hidden layer	Dataset size	Window size	Step length
ETTh1	12	50	17 420	24	12
ETTm1	12	100	69 680	6	3
Traffic	12	50	17 544	24	12
Exchange	6	50	7588	12	6
Solar	12	100	52 560	12	6
ECL	6	100	26 304	24	12
USLC	12	100	43 341	24	12
WSB	6	100	52 695	24	12
AirQuality	6	50	9358	12	6
ILI	6	20	1044	6	3

4.2. Experimental details

We choose different model parameters for different datasets. All models in our experiments are single-hidden-layer neural networks, and the sizes of the network hidden layer nodes for different datasets are set to 20, 50, and 100, respectively. More details are shown in Table 2. In our experiments, each univariate time series is converted into a matrices of $z \times d$ dimensions following Eq. (29). z is the size of dataset after preprocessing, and d can represent the number of nodes in input/output layer. The model used for time series prediction is a sequential learning process in our experiments. Therefore, we retain the last 48 observations from the dataset after preprocessing to test the predictive performance of the model. For RVFL-LSTM, the hyperparameter $C = 0.5$ is empirical, and the value m of $\boldsymbol{M}$ in Eq. (49) is $d/2$. The learning rate r during the gradient descent process in RVFL-LSTM is 0.1 and in LSTM is 0.001. To ensure a fair comparison, the number of layers in LSTM and OS-ELM is in consistent with RVFL, and the number of hidden layer nodes also remains the same. In LSTM, the epochs is 300, and the minimum loss threshold for early stopping is 0.0001.

We use two evaluation metrics, including Mean Squared Error (MSE) and Mean Absolute Error (MAE). All the datasets have a prediction horizon of $H = \{12, 24, 48\}$, which means that the larger the horizons, the harder the prediction tasks. All experiments are conducted three times, and the average MSE/MAE of the predicted results is obtained.

4.3. Results and analysis

The testing results under different prediction horizon are summarized in Table 3. The datasets contain multiple variables; however, our experimental results focus on univariate time series forecasting tasks. The specific univariate time series corresponding to the results shown in Table 3 are as follows: "oil temperature" for ETT dataset, the Australian exchange rate for Exchange dataset, the first feature for Traffic, Solar, and ECL datasets, "Hourly Wet-Bulb Temperature" for USLC dataset, "Tlog (degC)" for WSB dataset, "T" for AirQuality dataset, and "WEIGHTED ILI" for ILI dataset.

As shown in Fig. 6, it can be observed that RVFL-LSTM shows the best performance on the selected datasets, OS-ELM performs poorly on time series prediction tasks, and the results of LSTM are intermediate between these two methods. Besides, their predict error rises smoothly and slowly within the growing prediction horizon. Moreover, Table 3 shows that, the RVFL-LSTM gives the best results (wining-counts in the last column), while LSTM is close to the RVFL-LSTM, and both LSTM and RVFL-LSTM perform much better than OS-ELM.

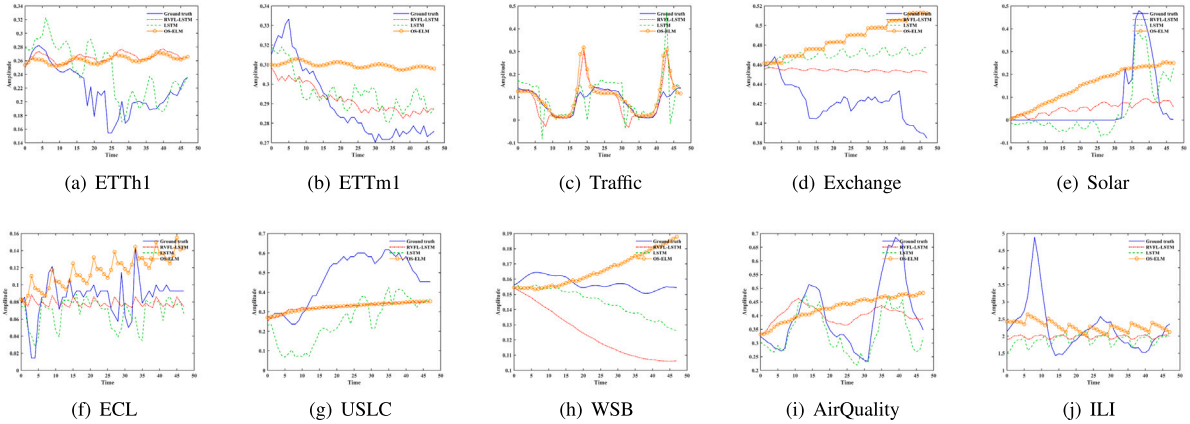
We note that RVFL-LSTM performs the best results in $H = 48$. Although RVFL-LSTM does not perform as well as LSTM on the Solar and AirQuality datasets, its performance outperforms OS-ELM. Fig. 6(d) shows that there are a large deviations between the predicted values and the observations. The relatively better performance of the RVFL-LSTM on the Exchange dataset may be due to the fact that it captures more short-term patterns. On Solar, ECL and AirQuality datasets, the performance between RVFL-LSTM and LSTM is different, though these datasets have many repetitive long- and short-term patterns. On the ILI dataset, all models yield relatively poor results.

Overall, the three methods have their respective strengths and weaknesses on different datasets. However, LSTM and RVFL-LSTM can capture both long-term and short-term patterns of time series, but they have different performance. To delve into the reasons, we construct two simulated time series datasets, L1 and L2, based on the sine function. The visualizations of L1 and L2 are depicted in Fig. 7. Subsequently, we apply LSTM and RVFL-LSTM for time series prediction on L1 and L2, respectively. The corresponding MSE values are presented in Table 4.

Table 4 reveals that RVFL-LSTM attains remarkable prediction accuracy for both L1 and L2. Based on the experimental results, we can speculate to some extent that RVFL-LSTM outperforms LSTM in simple time series with more repetitive long- and short-term patterns. This phenomenon can be attributed to the intrinsic nature of complex

Table 3
Prediction results comparison under different prediction lengths $H = \{12, 24, 48\}$.

Dataset			ETTh1	ETTm1	Traffic	Exchange	Solar	ECL	USLC	WSB	AirQuality	ILI	Count
LSTM	12	MSE	0.00120	**0.00009**	0.00114	0.00047	**0.00013**	0.00146	0.01526	**0.00005**	**0.00182**	2.66538	22
		MAE	0.02938	**0.00813**	0.02220	0.01810	**0.00931**	0.02916	0.10509	**0.00638**	**0.03026**	1.38858	
	24	MSE	0.00154	0.00017	**0.00186**	0.00161	**0.00034**	0.00091	0.06866	0.00006	**0.00231**	1.37485	
		MAE	0.03184	0.01035	**0.02597**	0.03496	**0.01372**	0.02272	0.22183	0.00669	**0.03511**	0.81209	
	48	MSE	**0.00182**	0.00042	0.00569	0.00235	**0.00947**	0.00092	0.08420	**0.00012**	**0.01243**	0.77285	
		MAE	**0.03178**	0.01534	0.03444	0.04400	**0.06100**	0.02324	0.25711	**0.00926**	**0.07700**	0.57545	
OS-ELM	12	MSE	0.00014	0.00013	**0.00006**	0.00060	0.00222	0.00164	**0.00088**	0.00006	0.00332	**1.08719**	14
		MAE	0.00991	0.00939	**0.00637**	0.01967	0.04026	0.02682	**0.02366**	0.00738	0.04461	**0.74059**	
	24	MSE	**0.00083**	0.00031	0.00387	0.00261	0.00995	0.00111	0.01721	**0.00005**	0.00531	**0.79696**	
		MAE	**0.02181**	0.01482	0.03076	0.04362	0.08514	0.02417	**0.09538**	**0.00636**	0.06233	**0.68922**	
	48	MSE	0.00289	0.00077	0.00371	0.00585	0.02251	0.00186	0.03432	0.00031	0.01410	**0.51506**	
		MAE	0.04433	0.02483	0.03176	0.06811	0.12911	0.03569	0.15707	0.01440	0.09926	0.54835	
RVFL-LSTM	12	MSE	**0.00005**	0.00026	0.00162	**0.00017**	0.00038	**0.00118**	0.00164	0.00028	0.00707	2.02854	24
		MAE	**0.00651**	0.01377	0.02685	**0.01176**	0.01716	**0.02531**	0.03295	0.01510	0.07437	1.13965	
	24	MSE	0.00096	**0.00016**	0.00307	**0.00083**	0.00116	**0.00072**	**0.01700**	0.00059	0.00489	1.06429	
		MAE	0.02190	**0.01020**	0.03148	**0.02467**	0.03013	**0.01943**	0.09801	0.02245	0.06038	0.69912	
	48	MSE	0.00334	**0.00015**	**0.00308**	**0.00119**	0.01713	**0.00060**	**0.03333**	0.00129	0.01318	0.58688	
		MAE	0.04716	**0.01067**	**0.03141**	**0.03073**	0.08321	**0.01902**	**0.15651**	0.03355	0.09045	**0.49282**	



(a) ETTh1 (b) ETTm1 (c) Traffic (d) Exchange (e) Solar

(f) ECL (g) USLC (h) WSB (i) AirQuality (j) ILI

Fig. 6. The time series prediction results.

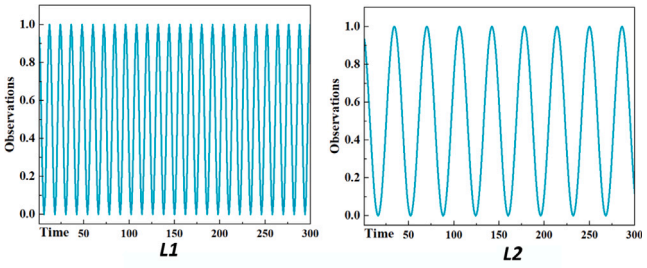


Fig. 7. Simulated time series L1 and L2.

Table 4
The prediction MSE of RVFL-LSTM and LSTM on L1 and L2.

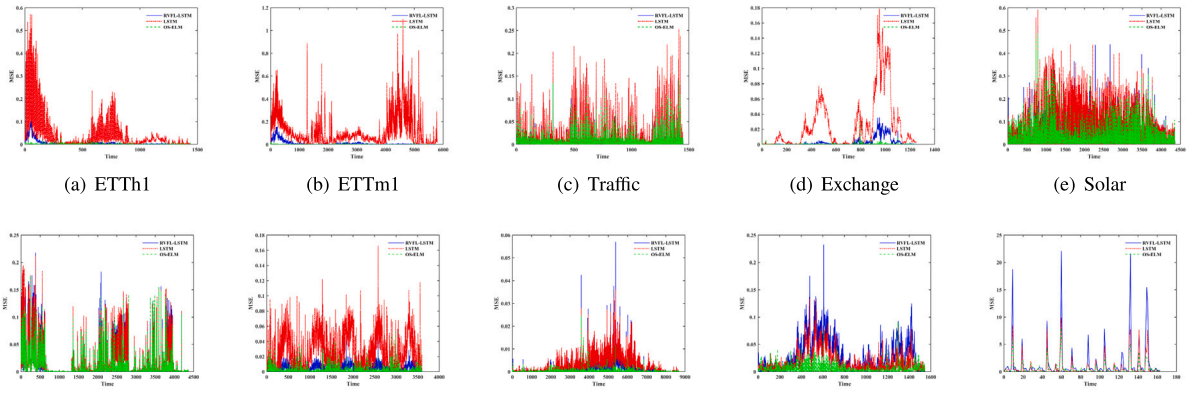
step	Method	L1	L2
12	LSTM	0.0019	0.519
	RVFL-LSTM	**0**	**0.0121**
24	LSTM	0.0029	0.406
	RVFL-LSTM	**0**	**0.0096**
48	LSTM	0.0043	0.324
	RVFL-LSTM	**0**	**0.0171**

time series, which often involve more sophisticated long-term or short-term patterns and nonlinearities. The gating mechanism and memory units inherent in LSTM enable it to effectively capture and utilize these intricate patterns, thereby contributing to improved prediction accuracy.

4.4. The long- and short-term patterns on the resulting model

It is mentioned in Section 3 that RVFL-LSTM can capture both the long- and short-term patterns, and the updating of $\boldsymbol{W}$ and $\boldsymbol{b}$ in RVFL-LSTM may result in the forgetting of old observations. In this subsection, a visualized analysis of these patterns and the forgetting phenomenon in the resulting RVFL-LSTM are provided. Fig. 8 shows the MSE for each window on the training data using the resulting models.

A definition is given for the forgetting in RVFL-LSTM, that is, the trained model cannot fit the training data perfectly. Specifically, when the training data is inputted into the resulting model, the fitting performance of the earlier observations in the time series tends to degrade. It is essential to highlight that the forgetting is fundamentally different from the underfitting of the model. The similar forgetting phenomenon is also observed in LSTM. In Fig. 8, we can observe that both LSTM and RVFL-LSTM hava larger MSEs for the early training data, while OS-ELM has low MSE on the entire training dataset. On Soalr (Fig. 8(e)) and ECL (Fig. 8(f)) datasets, the relatively large MSE of OS-ELM on the training dataset may be due to the fact that the model does not fit these observations well. In addition, LSTM, OS-ELM and RVFL-LSTM show relatively poor results on ILI dataset (Fig. 8(j)).



(a) ETTh1 (b) ETTm1 (c) Traffic (d) Exchange (e) Solar

(f) ECL (g) USLC (h) WSB (i) AirQuality (j) ILI

Fig. 8. The error of resulting model on the training set.

Table 5
The Comparison of training times (s).

Datasets	LSTM	OS-ELM	RVFL-LSTM
ETTh1	93.37	0.02	12.36
ETTm1	388.02	0.19	98.41
Traffic	81.12	0.02	12.65
Exchange	65.98	0.03	2.96
Solar	260.99	0.09	75.30
ECL	214.85	0.06	20.49
USLC	243.76	0.08	60.89
WSB	208.69	0.16	74.22
AirQuality	114.42	0.03	3.59
ILI	42.64	0.01	0.22

The short-term partterns is that the model have more importance of the new data during the sequential learning. It can be seen in Fig. 8, the RVFL-LSTM model obtains a low error for late observations in the time series on most selected datasets, which indicates that the RVFL-LSTM model is effective in capturing short-term patterns. The long-term patterns are that, the model maintains good fitting performance on the entire training dataset during the sequential learning process. Although RVFL-LSTM exhibits forgetting, it does not lose its fitting capability for early observations and only has a slight increase in MSE, as shown in Fig. 8(a), Fig. 8(b), Fig. 8(d), Fig. 8(g) and Fig. 8(h). The model can remember the previous data, it indicates that RVFL-LSTM can effectively capture long-term patterns in the time series.

If short-term patterns of the RVFL-LSTM is disregarded (i.e., the parameters $\boldsymbol{W}$ of the model are no longer adjusted when the new data arriving), this training approach resembles OS-ELM. The sequence learning capability of OS-ELM has already been demonstrated in Refs. [25, 26]. However, the results in our paper show that the learning ability of OS-ELM performs poorly on the selected time series datasets.

4.5. Time comparison

We compare the training time which is shown in Table 5. It can be observed that the OS-ELM has significantly shorter training times than LSTM or RVFL-LSTM. The training time of RVFL-LSTM is also much shorter than LSTM. Compared to OS-ELM, RVFL-LSTM has a longer training time, because it employs the BP algorithm to adjust the input weights. The training time is increasing due to the BP algorithm.

The performance of RVFL-LSTM is close to or even better than LSTM on some particular time series forecasting tasks, and the training time of RVFL-LSTM is shorter. We can conclude that RVFL-LSTM combines the performance of LSTM in time series prediction with the fast training speed of OS-ELM. It is particularly valuable for applications that require fast training and accurate deployment of models.

4.6. Discussion

In this study, we evaluate the performance of RVFL-LSTM on 10 public benchmark datasets covering six mainstream time series prediction applications. It is crucial to acknowledge that the difficulty of prediction may vary in different types of time series data. Similarly, different datasets can yield different results even when their type is the same.

The experimental results demonstrate that LSTM, OS-ELM and our proposed RVFL-LSTM have their respective strengths and weaknesses on different time series prediction tasks. RVFL-LSTM obtains satisfactory performance on the selected datasets in shorter training times. LSTM also achieves comparable performance, but the training time cost is too high. In contrast, OS-ELM does not achieve satisfactory performance. The simulation experiments show that RVFL-LSTM is more suitable for relatively simple datasets with more repetitive long- and short-term patterns, and this is a limitation of our proposed method.

However, the key contribution in this paper is that RVFL-LSTM can capture both long-term and short-term patterns in time series with a simple network structure. Meanwhile, it can be seen that both RVFL-LSTM and LSTM exhibit the forgetting of old observations, and OS-ELM treats all training data equally, which may be the reason for the poor performance on the selected time series prediction tasks.

5. Conclusion

This paper proposes an innovative time series prediction model named RVFL-LSTM, based on the moving auto-regression task and RVFL. The performance of RVFL-LSTM is evaluated on 10 widely used datasets, and the experimental results demonstrate that the proposed RVFL-LSTM model achieves more efficiency compared to LSTM, while the model performance is comparable. Additionally, compared with OS-ELM, RVFL-LSTM can capture long- and short-term patterns in time series and shows better performance on some particular time series prediction tasks. Regarding future work, we will continue to focus on time series prediction tasks and develop more efficient models. On the one hand, we would like to improve the model to achieve higher prediction accuracy, on the other hand, we will try to model multivariate time series using RVFL.

CRediT authorship contribution statement

Qiang Liu: Writing – review & editing, Writing – original draft, Visualization, Investigation. **Qin Wang:** Visualization, Validation, Software, Data curation. **Xizhao Wang:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grants (No. 62376161 and No. U24A20322); and the Stable Support Project of Shenzhen (Project No. 20231122124602001).

Data availability

Data will be made available on request.

References

[1] S. Chen, J. Hwang, Temperature prediction using fuzzy time series, IEEE Trans. Syst. Man Cybern. B 30 (2) (2000) 263–275.
[2] R. Salcedo, C. Ferraz, F. Martins, Time-series analysis of air pollution data, Atmos. Environ. 33 (15) (1999) 2361–2372.
[3] Q. Tan, M. Ye, A. Ma, B. Yang, T. Yip, G. Wong, P.C. Yuen, Explainable uncertainty-aware convolutional recurrent neural network for irregular medical time series, IEEE Trans. Neural Netw. Learn. Syst. 32 (10) (2020) 4665–4679.
[4] Z. Cao, C. Lin, Inherent fuzzy entropy for the improvement of EEG complexity evaluation, IEEE Trans. Fuzzy Syst. 26 (2) (2017) 1032–1035.
[5] L. Bai, L. Cui, Z. Zhang, Y. Xu, E. Hancock, Entropic dynamic time warping kernels for co-evolving financial time series analysis, IEEE Trans. Neural Netw. Learn. Syst. (2020).
[6] O.C. Yolcu, U. Yolcu, A novel intuitionistic fuzzy time series prediction model with cascaded structure for financial time series, Expert Syst. Appl. 215 (2023) 119336.
[7] G. Box, G. Jenkins, G. Reinsel, G. Ljung, Time Series Analysis: Forecasting and Control, John Wiley & Sons, 2015.
[8] R. Bollerslev, D. Nelson, ARCH models, Handb. Econometr. 4 (1994) 2959–3038.
[9] A. Nielsen, Practical Time Series Analysis: Prediction with Statistics and Machine Learning, O'Reilly Media, 2019.
[10] J.L. Elman, Finding structure in time, Cognit. Sci. 14 (2) (1990) 179–211.
[11] S. Hochreiter, Untersuchungen zu dynamischen neuronalen Netzen, Diploma, Technische Universität München 91 (1) (1991) 31.
[12] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[13] A. Graves, Long short-term memory, Supervised Seq. Label. Recurr. Neural Netw. (2012) 37–45.
[14] R. Li, Y. Hu, Q. Liang, T2F-LSTM method for long-term traffic volume prediction, IEEE Trans. Fuzzy Syst. 28 (12) (2020) 3256–3264.
[15] T. Guo, T. Lin, N. Antulov-Fantulin, Exploring interpretable LSTM neural networks over multi-variable data, in: International Conference on Machine Learning, PMLR, 2019, pp. 2494–2504.
[16] S. Kumar, S. Rai, R. Singh, S. Pal, ELM variants comparison on applications of time series data forecasting, in: 2016 International Conference on Advances in Computing, Communications and Informatics, ICACCI, IEEE, 2016, pp. 1404–1409.
[17] Y.-H. Pao, Y. Takefuji, Functional-link net computing: theory, system architecture, and functionalities, Computer 25 (5) (1992) 76–79.
[18] W. Cao, X. Wang, Z. Ming, J. Gao, A review on neural networks with random weights, Neurocomputing 275 (2018) 278–287.
[19] G. Huang, L. Chen, C. Siew, Universal approximation using incremental constructive feedforward networks with random hidden nodes, IEEE Trans. Neural Netw. 17 (4) (2006) 879–892.
[20] G. Huang, Q. Zhu, C. Siew, Extreme learning machine: theory and applications, Neurocomputing 70 (1–3) (2006) 489–501.
[21] G. Huang, Q. Zhu, C. Siew, Extreme learning machine: a new learning scheme of feedforward neural networks, in: 2004 IEEE International Joint Conference on Neural Networks (IEEE Cat. No. 04CH37541), Vol. 2, Ieee, 2004, pp. 985–990.
[22] A. Grigorievskiy, Y. Miche, A. Ventelä, E. Séverin, A. Lendasse, Long-term time series prediction using OP-ELM, Neural Netw. 51 (2014) 50–56.
[23] Z. Ma, Q. Dai, Selected an stacking ELMs for time series prediction, Neural Process. Lett. 44 (2016) 831–856.
[24] J. Xue, S. Zhou, Q. Liu, X. Liu, J. Yin, Financial time series prediction using $\ell_{2,1}$ RF-ELM, Neurocomputing 277 (2018) 176–186.
[25] G. Huang, N. Liang, H. Rong, P. Saratchandran, N. Sundararajan, On-line sequential extreme learning machine, Comput. Intell. 2005 (2005) 232–237.
[26] N. Liang, G. Huang, P. Saratchandran, N. Sundararajan, A fast and accurate online sequential learning algorithm for feedforward networks, IEEE Trans. Neural Netw. 17 (6) (2006) 1411–1423.
[27] X. Zhang, H. Wang, Selective forgetting extreme learning machine and its application to time series prediction, Acta Phys. Sin. 60 (8) (2011) 68–74.
[28] J. Lim, S. Lee, H. Pang, Low complexity adaptive forgetting factor for online sequential extreme learning machine (OS-ELM) for application to nonstationary system estimations, Neural Comput. Appl. 22 (2013) 569–576.
[29] Z. Ma, G. Luo, D. Huang, Short term traffic flow prediction based on on-line sequential extreme learning machine, in: 2016 Eighth International Conference on Advanced Computational Intelligence, ICACI, IEEE, 2016, pp. 143–149.
[30] J.-M. Park, J.-H. Kim, Online recurrent extreme learning machine and its application to time-series prediction, in: 2017 International Joint Conference on Neural Networks, IJCNN, IEEE, 2017, pp. 1983–1990.
[31] G.U. Yule, An Introduction to the Theory of Statistics, C. Griffin, 1927.
[32] C. Rao, K. Mitra, Further contributions to the theory of generalized inverse of matrices and its applications, Sankhyā (1971) 289–300.
[33] E.K. Chong, S.H. Żak, An Introduction to Optimization, vol. 75, John Wiley & Sons, 2013.
[34] J. Beran, Y. Feng, S. Ghosh, R. Kulik, Long-memory processes, 2013.
[35] E. De Faria, M.P. Albuquerque, J. Gonzalez, J. Cavalcante, M.P. Albuquerque, Predicting the Brazilian stock market through neural networks and adaptive exponential smoothing methods, Expert Syst. Appl. 36 (10) (2009) 12506–12509.
[36] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, No. 12, 2021, pp. 11106–11115.
[37] G. Lai, W.-C. Chang, Y. Yang, H. Liu, Modeling long-and short-term temporal patterns with deep neural networks, in: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, 2018, pp. 95–104.
[38] Y. Liu, H. Wu, J. Wang, M. Long, Non-stationary transformers: Exploring the stationarity in time series forecasting, Adv. Neural Inf. Process. Syst. 35 (2022) 9881–9893.
[39] N. Agarwal, N. Brukhim, E. Hazan, Z. Lu, Boosting for dynamical systems, 2019, arXiv preprint arXiv:1906.08720 194.